

Maximum-Shift String Matching Algorithms

Hakem Adil Kadhim

School of Computer Sciences,
Universiti Sains Malaysia, Penang, Malaysia
and
College of Engineering,
University of Kufa,
Kufa, Iraq.

hakima.kadhim@uokufa.edu.iq

Abstract— The string matching algorithms have broad applications in many areas of computer sciences. These areas include operating systems, information retrieval, editors, Internet searching engines, security applications and biological applications. Two important factors used to evaluate the performance of the sequential string matching algorithms are number of attempts and total number of character comparisons during the matching process. This research proposes to integrate the good properties of three single string matching algorithms, Quick-Search, Zuh-Takaoka and Horspool, to produce hybrid string matching algorithm called Maximum-Shift algorithm. Three datasets are used to test the proposed algorithm, which are, DNA, Protein sequence and English text. The hybrid algorithm, Maximum-Shift, shows efficient results compared to four string matching algorithms, Quick-Search, Horspool, Smith and Berry-Ravindran, in terms of the number of attempts and the total number of character comparisons.

Keywords—component; Hybrid String Matching; Quick-Search; Zuh Takaoka; Horspool, Arabic String Matching Systems

I. INTRODUCTION

String pattern matching is fundamental and central to many computer applications such as text processors, Internet-based search engines and computer security. It is also used in indexing algorithms, search algorithms and bioinformatics algorithms.

A key concept of string matching is identifying the existence of a pattern of m characters ($P = P_1, \dots, P_m$) in a very much longer text string $T (T_1, \dots, T_n)$. The generic behavior of string matching algorithms is based on a search window. Search window is defined as a virtual mechanism representing the part of the text that is compared with the given pattern and has a length generally equal to m , where m is the size of the given pattern. The matching of an input text starts by aligning the extreme leftmost positions for both the window and the pattern. The individual comparison is between the corresponding characters of the window and the pattern. Each event of characters comparison is called "attempt". The window at each attempt is shifted to the right of the text irrespective of a match or a mismatch between

NurAini AbdulRashid
School of Computer Sciences,
Universiti Sains Malaysia
Penang, Malaysia
nuraini@cs.usm.my

the pattern and the corresponding character in the window. The basic way of comparing two strings is to compare the first m -characters of the text (search window) and the pattern, after a match or a mismatch; the pattern is shifted by one character in the rightmost direction of the text. This algorithm is considered as the oldest and most common algorithm in computer science and called Brute-Force (BF).

In this paper we present a hybrid algorithm based on the good features of three string matching algorithms. In the next section we presents the variations of Boyer-Moore algorithms, which are used in this research and then followed by the detail of the hybrid algorithm. We also include current hybrid string matching algorithms. The detail analysis of the proposed algorithm and the results of the experiment are then presented.

II. RELATED STRING MACHING ALGORITHMS

Generally, string matching problem is divided into two independent groups: approximate string matching and exact string matching [1]. Exact string matching can be defined as a technique of finding all the exact occurrences of a given pattern $P = P[0] P[1] \dots P[m-1]$ of length m in a large text $T = T[0]T[1] \dots T[n-1]$ of length n , where $m, n > 0$ and $m \leq n$. Both P and T belong to the same alphabet Σ . Most string matching algorithms have two phases, which are preprocessing and searching phase. Preprocessing phase preprocess the pattern characters to determine the distance that the pattern should shift, while searching phase uses the information to find the pattern occurrences within a long text.

One of the most popular algorithms in this field is Boyer-Moore (BM) Algorithm. BM algorithm is based on three smart ideas, which are the right-left character comparisons, bad character heuristics and good suffix heuristics. Right-left character comparisons are useful in collecting more information about the scrutinized characters and the information is used during the searching phase [2]. While bad character heuristics concept is adopting the pattern shifting based on the text character that causes the mismatch event between the text and the pattern strings. Finally, good suffix heuristics is used to shift the pattern to the right side of the text based on the similar suffixes between the pattern

978-1-4799-0059-6/13/\$31.00 ©2014 IEEE.

and the text. Due to the efficient behavior of BM algorithm in many applications, many variations have been derived from original BM algorithm to fit many purposes.

One of the variant of BM algorithm is the Horspool (HP) algorithm [3], which avoided the uses of good suffix heuristics during it process. The preprocessing phase is based only on the preprocessed value of text character that corresponds to the rightmost character of the pattern, which is used to shift the pattern to the next position along the text. This operation is repeated on both for the match or a mismatch pattern and run all over the text. Quick-Search (QS) [4] uses only the bad-character heuristic of BM with slight modification in the pattern shift. The pattern shift is based on the character that follows the rightmost character of the text window when there is a match or a mismatch. In practice, QS is very fast in searching short and long patterns and is easy to implement. Another variant of BM is Zuh-Takaoka [5] where the difference is centered in the preprocessing phase, where the pattern is shifted based on the two consecutive characters of the rightmost text window. If these two characters existed in the pattern, the pattern is shifted to align the text at these two characters. But, if the two characters did not occurred in the pattern, the algorithm shifted the pattern by m characters for the next attempt.

There are many existing algorithms that applied Quick-Search and Zhu-Takaoka algorithms. The basic idea behind Berry-Ravindran algorithm is to increase the proportion of skipping the search window along the text [6]. Quick-Search algorithm considers the character that follows the rightmost character of the search window to shift the pattern. Zhu-Takaoka algorithm uses the two consecutive rightmost characters of the search window to calculate the shift value. In 2006, Thathoo proposed another form of a hybrid algorithm TVSBS algorithm [7] Briefly, the idea of the TVSBS algorithm is to inspect the first rightmost character of both the window of the text and the pattern, if the characters match, the same inspection operation is repeated but for the first leftmost characters. If these corresponding characters are also matched, the reset characters in the pattern and the window are compared from right to left until the pattern is matched or mismatched. Once a match or a mismatch occurred, the pattern is immediately shifted to the right of the text based on the shift value. The shift value of TVSBS algorithm is provided by preprocessing phase, which uses the Berry- Ravindran (BR) bad character function with one-dimensional array to store the shift values.

III. THE PROPOSED MAXIMUM-SHIFT ALGORITHM

The essential aim of this study is solving the string-matching problem by integrating three existing single algorithms to produce an efficient sequential hybrid algorithm. The proposed hybrid algorithm, Maximum-Shift algorithm, consists of three stages, as described in the following three subsections:

A. Preprocessing Stage:

Generally, the preprocessing phase is used to preprocess the pattern characters to be useable in the searching phase in order to reduce the number of attempts and the number of character comparisons. We use the preprocessing function of two single algorithms, which are QS and ZT algorithm, to design the preprocessing stage of our proposed hybrid algorithm. The QS preprocessing function provides maximum shift value, $m+1$, when a match or a mismatch occurs and the character that follows the rightmost character of the text window does not occur in the pattern. On the other hand, it provides minimum shift value when the character follows the text window equals to the rightmost character in the pattern. See the equation below:

$$qsBc[c] = \begin{cases} \min\{i: 0 < i \leq m \text{ and } x[m-i] = c\} & \text{if } c \text{ occur in } x \\ m+1 & \text{Otherwise} \end{cases}$$

Similarly, the ZT preprocessing function provides maximum shift value; m , when a match or a mismatch occurred and the two consecutive rightmost characters in the text window did not occurred in the pattern. But the ZT preprocessing function provides minimum shift value when the two consecutive rightmost characters occurred near the positions in the rightmost pattern. The ZT preprocessing function fills by using the equation below:

For $a, b \in \Sigma$

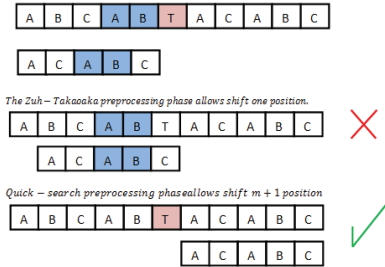
$$ztBc[a, b] = k \Leftrightarrow \begin{cases} k = m & x[0] \neq b \text{ and } ab \text{ does not occur in } x[0..m-2], \\ k = m-1 & x[0] = b \text{ and } ab \text{ does not occur in } x[0..m-2], \\ k \leq m-2 & \text{and } x[m-k-2..m-k-1] = ab \text{ and } ab \text{ does not occur in } x[m-k-1..m-2] \end{cases}$$

Due to the reverse behavior of these two preprocessing functions, the basic idea of our proposed solution is harnessing the advantages of each function to avoid the disadvantages of another. The values produced from this stage is passed as inputs to the second stage of our proposed design, which is the Max-shift stage, to choose the shift value that will be used to shift the pattern during the searching phase.

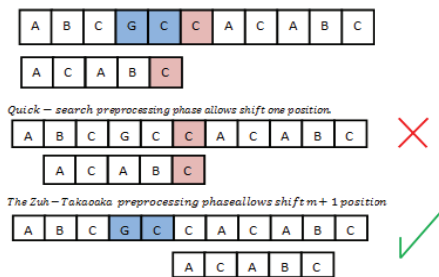
B. Max-Shift Stage:

This stage is considered as a link between the preprocessing phase and searching phase. In this stage, after receiving the two shift values from the QS preprocessing phase and ZT preprocessing phase, there are two cases that have to be considered in order to provide maximum shift value to the pattern. These two cases are:

1) If the two consecutive rightmost characters of the window occur near the rightmost pattern, and the character following the rightmost character in the window does not occur in the pattern or occurs in a position further than the two consecutive characters, then the proposed Max-shift stage takes the QS shift value instead of ZT shift value to shift the pattern. See example below.



2) If the character following the rightmost character of the window equals exactly to the rightmost character of the pattern, and the two consecutive rightmost characters in the window do not occur in the pattern or occur in a position further than occurring the character that follows the rightmost character of the window, then the proposed Max-shift stage takes the ZT shift value instead of QS shift value to shift the pattern. See example below.



These two cases can be verified by taking the maximum value of the two shift values of both the existing preprocessing phases at each time of matching or mismatching the pattern along the text. The Max-Shift function reduced the number of attempts as well as the total number of character comparisons significantly.

C. Searching Phase:

In our proposed hybrid algorithm, we used the idea of left to right comparison of QS algorithm as well as the Horspool's right-left initial step with slight modification on the latter. This modification involves checking the first two rightmost characters of the window and the pattern as an initial step instead of checking only one character. The proposed searching phase significantly reduces the number of character comparisons at only one attempt by achieving 2 character comparisons to know the mismatch. On the other hand, Horspool needs 7 comparisons and Quick-Search consumes 6 character comparisons to discover the pattern mismatch during one attempt (Shown in Figure.1.)

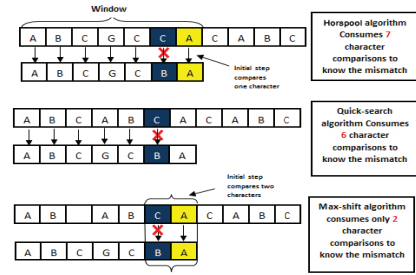


Figure 1. Proposed Hybrid Algorithm

IV. MAX- WORKING EXAMPLE

Arabidopsis thaliana is a genome that is composed of 27,242 gene sequences and it is divided over five chromosomes (CHR_I to CHR_V) [7]. To test the proposed hybrid algorithm, a small portion of a nucleotide sequence of a gene (only 47 nucleotides) from Chromosome I (CHR_I) has been chosen [7] According to the FASTA format, the sequence that is taken from the gene sequence from index 32854 to 32901 is:

$y = \text{ATCTAACATCATAACCCTAATTGGCAGAGAGAGAATCAATCGAATCA}$

While the pattern that is used to test the proposed algorithm is: $x = \text{GCAGAGAG}$

$n = 47$, $m = 8$, where n is the length of the small portion of the gene and m is the length of the chosen pattern.

Preprocessing phase: The preprocessing phase of alphabet size $\sigma = 4$ is built by constructing the QS and ZT bad character tables to the input pattern as shown in Table.1 and Table. 2.:

Table. 1. Shift values for σ given by ztBc function

	A	C	G	T
A	8	8	4	8
C	5	8	7	8
G	3	6	7	8
T	8	8	7	8

Table. 2. Shift values for $\sigma = 4$ given by the qsBc function

P	A	C	G	T
QsBc	2	7	1	9

The proposed searching phase of the new algorithm inspects the first and second rightmost characters that are positioned in the text and the pattern. The mismatch event occurred at the first comparison operation for the first attempt. Therefore, the proposed preprocessing function, in this case, should choose the maximum value of either the QS shift value or ZT shift value. The maximum shift value has been chosen is 9, because the QS shift function is based on letter T (the character that follows the rightmost character of the window) and the letter of T according to Table 1 equals to 9, while the ZT shift function is based on the letters CA and the value of CA according to Table 2 is equivalent to 5. Therefore, the proposed shift function selects 9 as a maximum shift value. The second, third, fifth, sixth and seventh attempts behaved equally when comparing the characters and shifting the pattern. The proposed algorithm in the fourth attempt ensured that the first and second rightmost characters are matched and then transferred directly to inspect the rest of the characters by starting from the beginning of the leftmost characters. Eventually, the proposed algorithm has decided that the pattern at the fourth attempt is matched with the given text after ensuring the exact match of all characters of pattern with the substring of the given text. According to the above example, the proposed algorithm inspected all the text characters and found one exact matching with 7 attempts and 17 characters comparison operation.

V. ANALYSIS

The preprocessing phase of the proposed hybrid algorithm (Maximum-Shift algorithm) consists of the preprocessing phase QS and ZT algorithm. Therefore, the time complexity of the preprocessing phase of the proposed hybrid algorithm is $O(m + \sigma^2)$. Where σ represents the alphabet size. On the other hand, the time complexity of the searching phase is divided into two cases as follows:

Worse case: to find the searching phase time complexity of the proposed algorithm in its worse case, we will assume that the text is $T = \text{"aaaaaaaaaaaaaa"}$ and the pattern is $P = \text{"aaaaa"}$. In this case, the proposed algorithm inspects all characters of pattern and what they are corresponding in the text window. The shift value in this case is equaled to 1 position at each time of matching event. Therefore the time complexity of the proposed algorithm in terms of the searching phase in its worse case is $O(nm)$.

Best-case: the text that is chosen to test the best-case of the proposed searching phase is $T = \text{"aaaaaaaaaaaaaa"}$, while the pattern is $P = \text{"bbbbbb"}$. The preprocessing phase of the

proposed algorithm is affected directly in the searching phase, where the shift value is always represented by the QS shift value, which is $(m+1)$. This is because of, in this case, the qsBc $[m+1]$ always greater than ztBc $[m-1]$ $[m]$. Therefore, the time complexity of the searching phase in its best case is calculated according the following equation: $O(n/(m+1))$.

VI. EVALUATION

There are three different types of data that are used to evaluate the proposed hybrid string-matching algorithm. These types involve DNA sequence with alphabet size equal to 4 letters ($\sigma=4$), Protein sequence with alphabet size equal to twenty letter ($\sigma=20$), and English text with over 100 alphabets involving all English language letters, numbers and symbols. The motives behind choosing these specific types of data are first; they are considered as the common benchmark datasets, which is presented as string matching applications. Second, they have different alphabet sizes, which, in turn, help to examine the proposed hybrid algorithm in various circumstances. The size of each data type that is used to evaluate the sequential version is 100MB. The algorithm is implemented on a personal computer with 2.0 GHz Intel Core (TM) 2 Duo processor and 4 GB of RAM. The operating system was Microsoft Windows 7 ultimate version of the 32-bit operation system. Microsoft visual studio 2008 is the editor that is used to write the codes and the visual C++ compiler compiles and runs the programs.

There are two common performance measures used in many works to evaluate the performance of string matching algorithms with different applications [1] and these measures are as listed below:

A. Number of Character Comparisons:

This factor refers to the actual comparisons, which happen between the characters of the text window and the pattern. The algorithm with less number of character comparisons is considered as an efficient algorithm with better performance.

B. Number of Attempt:

This factor considers the distances that the pattern has to shift along the whole given text. Obviously, whenever the number of attempts is less, the algorithm performance is better.

VII. RESULTS

As mentioned previously, four string matching algorithms are used to compare against the proposed hybrid algorithm, and these algorithms are: Quick-search, Horspool, Smith and Berry-Ravindran algorithm. Different types of datasets are used of the size of 100MB each to evaluate the sequential version of each algorithm. The patterns are chosen randomly from within each dataset and have

different lengths that ranges from short pattern lengths of 8 and 10 [4] to long pattern lengths, which are 20,30,40,50,60,70,80,90 and 100 characters [8]. The empirical results of computing the number of attempts and the number of characters comparison for DNA dataset are shown in Figure. 2 and Figure. 3.

The percentage of the improvements of the new hybrid algorithm compared to QS, Horspool, Smith and Berry-Ravindran algorithm are higher with the single algorithms compared to the hybrids algorithms for both the number of attempts and number of character comparison.

Figure.4 and Figure.5, show the results of the number of the attempts and the number of the characters comparison for Protein while Figure. 6 and Figure. 7 for English Text. The patterns of the results are the same as the one running on DNA data with slight decrease in improvements.

In general, Maximum-Shift algorithm has efficient preprocessing phase (QS+ZT) that helps to increase the distance that the pattern slides during the searching operation. Increasing the distance of pattern shifting leads to the decreasing of the number of attempts. Because of this property, the Maximum-Shift shows more stable behavior in terms of achieving fewer numbers of attempts for DNA, Protein and English text datasets. In addition, the modification of Horspool searching phase, which is used in our proposed hybrid algorithm and based on the property of inspecting two rightmost characters instead of one character, decreases the number of character comparisons during the searching phase. Consequently, the Maximum-Shift algorithm shows steadier action in terms of achieving fewer numbers of character comparisons compared to other algorithms.

VIII. CONCLUSION

String matching problems occupy a wide space in many computer science applications; this study presents an efficient solution to adopt these problems depending upon the algorithm's hybridization concept. There are three single exact string matching algorithms belong to the Boyer-Moore family (QS, ZT and BMH) which has been analyzed to identify their good properties and combine them in new hybrid algorithm called Maximum-Shift. The preprocessing phase of our proposed algorithm is built by combining the QS preprocessing function and ZT preprocessing function to take the maximum value in shifting the pattern as far distance as possible along the text. The searching phase of the proposed algorithm is based on Horspool searching phase with slight modification. The proposed searching phase, firstly, inspects two rightmost characters between the text window and the pattern instead of inspecting one character, as it takes place in Horspool algorithm. There are

two factors that are used in this work to evaluate the performance of the proposed hybrid algorithm, the number of attempt and the total number of character comparisons. For evaluating the proposed hybrid algorithm, there are three benchmark databases (DNA sequence, Protein sequence and English text data type) used with different pattern lengths are chosen randomly from within each database. The proposed Maximum-Shift algorithm showed better performance compared to four string matching algorithms (QS, Horspool, Smith and BR) by achieving fewer numbers of attempts and fewer numbers of character comparisons. This is because of the preprocessing phase of Maximum-Shift algorithm provides maximum shift value at each match or mismatch occurrence. The longer distance that the pattern is shifted along the text leads to significantly decrease the number of attempts during the searching operation. While using the modification of Horspool searching phase as a proposed searching phase for Maximum-Shift algorithm succeeded to decrease the total number of character comparisons during the matching operation.

IX. ACKNOWLEDGEMENT

We would like to acknowledge Ministry of Higher Education for supporting this publication under ERGS grant USM/Pkomp/6730074

X. REFERENCES

- [1] Navarro, G. and Raffinot, M., (2002). Flexible pattern matching in strings: practical on-line search algorithms for texts and biological sequences, Press Syndicate of The University of Cambridge.
- [2] Boyer, R. S. and Moore, J. S. (1977). A fast string-searching algorithm, *Communications of the ACM*, 20(10), 762-772.
- [3] Horspool, R. N. (1980). Practical fast searching in strings, *Software-Practice and Experience*, 10(6), 501 - 506.
- [4] Sunday, D. (1990). A very fast substring search algorithm, *Communications of the ACM*, 33(8), 132- 142.
- [5] Feng, Z., R. and Takaoka, T. (1987). On improving the average case of the Boyer-Moore string matching algorithm. *Journal of Information Processing* 10 (3): 173-177.
- [6] Berry, T., & Ravindran, S. (1999). A Fast String Matching Algorithm and Experimental Results. In *Stringology* (pp. 16-28).
- [7] Thathoo, R., Virmani, A., Lakshmi, S. S., Balakrishnan, N. and Sekar, K. (2006). TVSBS: A fast exact pattern-matching algorithm for biological sequences. *Journal of the Indian Academy of Science, Current Science*, 91(1): 47-53.
- [8] Naser, M. A. (2009). Parallel quick-skip search hybrid algorithm for exact string matching problem, Master's thesis, Universiti Sains Malaysia. Pusat Pengajian Sains Komputer.
- [9] NCBI site, ftp://ftp.ncbi.nih.gov/genomes/Arabidopsis_thaliana/CHR_I

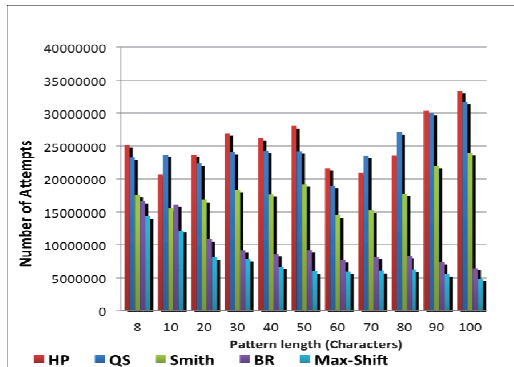


Figure 2: No Of Attempts (DNA)

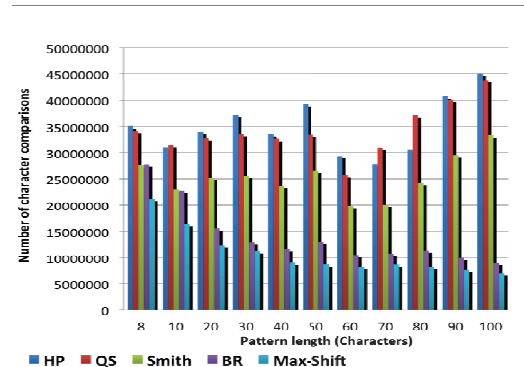


Figure 3: No of Character Comparison (DNA)

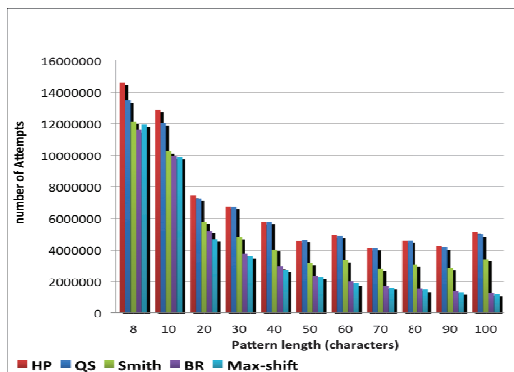


Figure 4: No of Attempts (Protein)

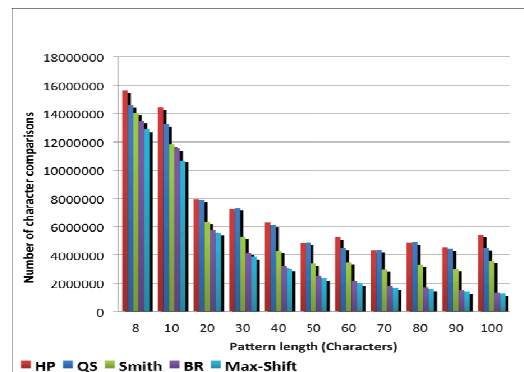


Figure 5: No of Character Comparison (Protein)

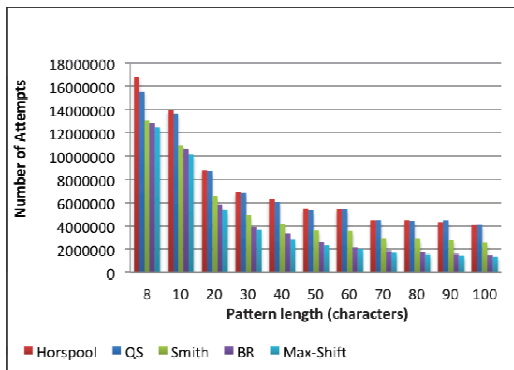


Figure 6: No of Attempts (English Text)

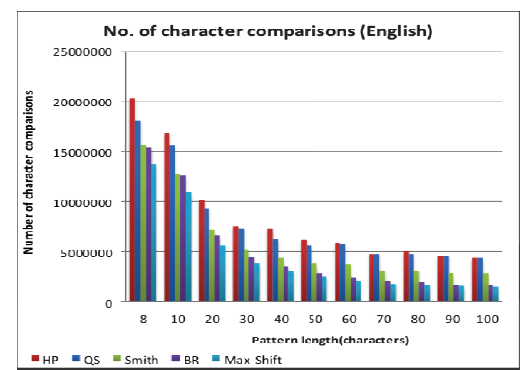


Figure 7: No of Character Comparison (English Text)